

mi_rutina_user.js

Pantalla del lado del alumno que muestra un resumen de TODA la rutina de una sola vista: todos los dias, y dentro de cada dia, todos sus bloques e items (de forma condensada). Es la pantalla 'overview' de la semana completa; desde aca se navega al detalle de un dia especifico (ver_dia_x_user.js) con el boton 'Ver dia'.

1. Obtencion del usuario y su rutina (JOIN simulado)

```
const { user } = useAuth();
if (!user) return null;
const routine = getRoutineById(user.id_rutina);
```

A diferencia de ver_datos_trainer.js y ver_dia_x_user.js (que reciben sus datos ya armados via route.params, porque dependen de a que alumno o dia especifico se navego), esta pantalla obtiene al usuario logueado directamente del AuthContext, porque siempre se trata del 'usuario actual', no de un dato que llega por navegacion. Con el campo user.id_rutina se busca la rutina completa en el mock routines.js mediante getRoutineById, simulando el JOIN que se haria en una base de datos real entre la tabla de usuarios y la de rutinas a traves de una foreign key.

El guard 'if (!user) return null' protege un caso puntual: durante la animacion de cierre de sesion, por un instante el componente puede seguir montado con 'user' ya en null. Sin este chequeo, la linea siguiente (user.id_rutina) rompería con un error de 'no se puede leer una propiedad de null'.

2. Alumno sin rutina asignada

getRoutineById puede devolver undefined si el alumno todavia no tiene una rutina asignada (por ejemplo, recién registrado). La pantalla contempla este caso mostrando un mensaje alternativo: {routine ? routine.name : 'Todavía no tenes una rutina asignada'}. Y el encadenamiento opcional en routine?.days.map(...) evita que la pantalla rompa si routine es undefined (en ese caso simplemente no renderiza ningun dia).

3. Jerarquia de render: dia -> bloque -> item

```
routine?.days.map((day) => (
  ...
  day.blocks.map((block, blockIndex) => (
    <Text>{`Bloque ${blockIndex + 1}`}</Text>
    block.items.map((item) => ( /* fila resumida */ ))
  ))
))
```

Tres niveles de .map() anidados, cada uno con su propio key (day.id, block.id, item.id) para que React pueda reconciliar la lista correctamente. Igual que en ver_dia_x_user.js, el titulo del bloque se calcula a partir de blockIndex en cada render, no se guarda en los datos: es coherente con que el campo 'name' de los bloques fue eliminado de routines.js, ya que un entrenador real nunca le pone nombre a un bloque individual.

4. Cada item: fila compacta, no una tarjeta completa

A diferencia de ver_dia_x_user.js (donde cada ejercicio es una tarjeta grande con botones de video e instrucciones), en esta pantalla cada item es una fila compacta con un icono a la izquierda y el texto

resumido a la derecha:

```
<View style={styles.item}>          {/* flexDirection: row */}
  <View style={styles.itemIcon}>      {/* circulo con icono */}
    <Text>{item.type === 'exercise' ? '■' : '■'}</Text>
  </View>
  <View style={styles.itemInfo}>      {/* flex:1, texto */}
    {item.type === 'exercise' ? (
      <>...nombre, series.reps.peso, comentarios...</>
    ) : (
      <Text>Descanso · {item.time} {item.unit}</Text>
    )}
  </View>
</View>
```

Esto es intencional: esta pantalla es para tener una vista rapida de toda la semana, no para entrenar con ella en la mano dia a dia (para eso esta `ver_dia_x_user.js`) — por eso la informacion esta condensada en una sola linea (`series · reps · peso`) en vez de las cajas separadas de datos que tiene la pantalla de detalle. El icono usa dos glifos Unicode distintos ('■' para ejercicio, '■' para descanso) dentro de un circulo con fondo ambar, dando una pista visual rapida del tipo de item sin necesidad de leer el texto.

5. Navegacion al detalle de un dia

```
onPress={() => navigation.navigate('ver_dia_x_user', { day })}
```

Le pasa el objeto 'day' completo (con sus blocks e items) como parametro de navegacion. Es el mismo patron de 'pasar el objeto ya armado' que se usa en toda la app (por ejemplo `buscar_usuario_trainer.js` pasando 'user' a `ver_datos_trainer.js`), evitando que la pantalla destino tenga que volver a buscar el dato por su cuenta.

6. Estilos

Mismo lenguaje visual que el resto de la app: los bloques ('block') usan el borde amarillo palido (#FFE082) que se repite en `ver_dia_x_user.js` y `editar_rutina_trainer.js`. El texto de descanso ('restText') es celeste (#66B8FF) para diferenciarse visualmente de los ejercicios de un vistazo. Los tamanios de texto de toda la pantalla fueron agrandados en dos pasadas sucesivas para mejorar la legibilidad general mientras se usa el celular.